StudyFlow : Cahier des charges v2.1 (Flutter + Dart)

Version 2.1 consolidée · Application de gestion de projet pour équipes étudiantes (type mini-Trello / Notion), avec formations et coaching

Rubrique	Valeur
Type	Application Flutter (web en priorité), exécutée en local
Technologie principale	Flutter + Dart (client) · Serverpod (serveur Dart)
Équipe	7 développeurs, 1 module chacun
Version du document	2.1 (remplace les versions 2.0 et 2.1 partielle)

Changements de la version 2.1

- L'ancien Module 7 (Statistiques et rapports) est intégré au **Module 4**, qui devient « Calendrier, échéances, statistiques et rapports ».
- Le **Module 7** devient « Formations et coaching » : catalogue de formations, inscriptions, présence, séances de coaching, comptes rendus, objectifs et compétences.
- Un nouveau rôle d'équipe, **Coach**, est ajouté (§4). La matrice des permissions, le modèle de données (§7), les contrats entre modules (§6.3), le planning, les risques et la recette sont mis à jour.
- Le dimensionnement de **18 j-h par développeur** est conservé. Les responsabilités transverses sont redistribuées (§8.2) : le Dev 4 prend le journal d'activité, le Dev 7 reprend les tests de bout en bout.
- Les modules 1, 2, 3, 5 et 6 gardent leur contenu fonctionnel ; seules leurs intégrations avec les modules 4 et 7 changent.
- **Correction** : le total d'entités est de **24** (et non 21) : 5 modules × 3 entités + 6 entités pour le M4 + 3 pour le M7.

1. Présentation du projet

1.1 Contexte

Les étudiants travaillent de plus en plus en groupe : projets de cours, projets tutorés, associations, hackathons. Leur organisation repose souvent sur un empilement d'outils mal reliés : un groupe WhatsApp pour discuter, un Google Drive pour les fichiers, un tableur ou des notes pour les tâches, et un calendrier personnel pour les échéances. Les informations se perdent, les responsabilités sont floues, et personne n'a de vue d'ensemble sur l'avancement.

Les outils professionnels (Trello, Notion, Jira, Asana) répondent à ce besoin, mais ils sont soit trop complexes, soit payants, soit inadaptés au contexte étudiant (notion de matière, de rendu, de soutenance, d'évaluation par un enseignant). De plus, la montée en compétences de l'équipe (ateliers, formations techniques, séances de coaching avec un enseignant, un mentor ou un ancien) s'organise hors de l'outil, par e-mail ou messagerie, sans lien avec les tâches du projet.

1.2 Problématique

Comment permettre à une équipe d'étudiants de planifier, répartir, suivre, communiquer et progresser en compétences autour d'un projet dans un seul outil simple, gratuit et adapté à leur réalité ?

1.3 Objectifs

Objectifs principaux

- Centraliser dans une seule application la planification, les tâches, les échéances, la communication et les fichiers d'un projet.
- Donner à chaque membre une vision claire de ce qu'il doit faire et pour quand.
- Donner à l'équipe (et à un enseignant référent) une vue objective de l'avancement, grâce aux statistiques et rapports intégrés au calendrier (Module 4).
- Organiser la montée en compétences : formations internes, séances de coaching, objectifs de progression et suivi des compétences (Module 7).

Objectifs pédagogiques (pour l'équipe de développement)

- Maîtriser Flutter : arbre de widgets, gestion d'état avec Riverpod, navigation déclarative avec go_router, interface adaptative (web, tablette, mobile).

- Maîtriser Dart 3 : null safety, classes scellées (sealed), enregistrements (records), programmation asynchrone et flux (Future, Stream).
- Concevoir un serveur en Dart avec Serverpod (endpoints, modèles, migrations) et exploiter une base de données relationnelle.
- Mettre en œuvre de l'authentification, de l'autorisation par rôle, du temps réel et de l'upload de fichiers.
- Travailler en équipe de 7 : Git, revue de code, intégration continue et conventions partagées.

1.4 Indicateurs de réussite

- Une équipe de 5 étudiants peut créer un projet, inviter ses membres, créer 20 tâches et les répartir en moins de 10 minutes sans aide.
- Un membre peut s'inscrire à une formation et demander une séance de coaching en moins de 2 minutes.
- Tous les parcours critiques sont utilisables sur ordinateur et sur un écran de 375 px de large (mobile).
- Les 7 modules sont intégrés dans une application unique, qui se lance en local en moins de 10 minutes à partir du dépôt.

2. Périmètre

2.1 Inclus (version 2.1)

- Inscription, connexion, gestion du profil, création d'équipes, invitations, rôles (dont Coach).
- Plusieurs projets par équipe, chacun avec son tableau Kanban.
- Création, édition, assignation et suivi de tâches détaillées.
- Calendrier des échéances (mois / semaine / liste), jalons, rappels, statistiques d'avancement, journal d'activité et rapports exportables (Module 4).
- Chat par équipe et par projet, en temps réel ; notifications dans l'application.
- Dépôt, organisation et consultation de fichiers.
- Formations (catalogue, inscriptions, présence, évaluation) et coaching (demandes de séances, comptes rendus, objectifs, compétences) (Module 7).

2.2 Exclu (hors périmètre de la version 2.1)

- Publication sur les magasins d'applications et notifications push natives.
- Appels audio / vidéo intégrés : une séance en ligne se fait via un lien externe saisi par le coach (Meet, Teams, Jitsi).
- Intégrations tierces (Google Calendar, GitHub, Slack), sauf export iCal simple.
- Facturation, abonnement, paiement (y compris formations ou coaching payants) et délivrance de certifications officielles.
- Édition collaborative de documents en temps réel ; mode hors-ligne.
- Mise en ligne de l'application (hébergement public) : StudyFlow s'exécute en local.

2.3 Plateformes cibles

Plateforme	Statut	Remarque
Flutter Web (Chrome, Edge, Firefox, Safari)	Cible principale	Plateforme de la recette et de la démonstration. Interface adaptative de 375 px à l'ordinateur.
Android (émulateur ou appareil)	Bonus	Même code Dart. À compiler et tester en semaine 7 si le temps le permet.
Windows, macOS, Linux (bureau)	Bonus	Même code Dart, sans travail dédié.
iOS	Hors périmètre	Nécessite un Mac et un compte développeur.

2.4 Évolutions possibles (bonus)

- Notifications par e-mail et résumé hebdomadaire.
- Mode sombre (Material 3 : deuxième ColorScheme).
- Diagramme de Gantt ; tableau de bord global multi-projets.
- Application Android installable et notifications push.
- Parcours de formation en plusieurs étapes avec badges.

3. Utilisateurs cibles

3.1 Personas

Persona	Profil	Besoin principal
Léa, cheffe de projet	Étudiante de L3, organise le projet de groupe	Répartir le travail, relancer, avoir une vue d'ensemble
Karim, contributeur	Étudiant dans l'équipe, travaille aussi à côté	Savoir clairement ce qu'il doit faire et pour quand ; combler ses lacunes techniques
Mme Dupont, enseignante référente et coach	Encadre plusieurs groupes	Suivre l'avancement sans microgérer ; animer des formations et des séances de coaching
Sami, étudiant multi-projets	Membre de 3 équipes différentes	Une vue unifiée de ses tâches, échéances, formations et séances

3.2 Parcours type

1. Léa crée un compte, puis crée l'équipe « Projet Web S5 » et invite 4 camarades par lien d'invitation, ainsi que Mme Dupont avec le rôle Coach.
2. Elle crée un projet « Application de réservation » avec un tableau Kanban (À faire, En cours, En relecture, Terminé).
3. Elle crée les tâches, les assigne, fixe des échéances.
4. Karim se connecte, voit ses tâches, déplace une carte vers « En cours », joint un fichier, commente.
5. Mme Dupont publie une formation « Introduction à Riverpod » ; Karim s'inscrit, elle apparaît dans son calendrier.
6. Karim demande une séance de coaching sur l'architecture ; Mme Dupont confirme un créneau et rédige ensuite un compte rendu avec des actions.
7. L'équipe discute dans le chat du projet.
8. La veille du rendu, Léa consulte le tableau de bord d'avancement dans le calendrier du projet et exporte un rapport PDF.

4. Rôles et permissions

4.1 Rôles au niveau d’une équipe

Rôle	Description
Propriétaire	Créateur de l’équipe. Tous les droits, dont la suppression de l’équipe et le transfert de propriété.
Administrateur	Gère les membres, les projets, les formations et les paramètres.
Membre	Crée et modifie des tâches, participe au chat, dépose des fichiers, s’inscrit aux formations, demande du coaching.
Coach	Enseignant référent, mentor ou ancien. Lecture de tout le projet, commentaires, animation de formations, gestion des séances de coaching. Ne modifie pas les tâches.
Observateur	Lecture seule. Peut commenter si autorisé.

4.2 Matrice des permissions

Action	Propriétaire	Admin	Membre	Coach	Observateur
Supprimer l’équipe	✔	✘	✘	✘	✘
Inviter / retirer un membre	✔	✔	✘	✘	✘
Changer le rôle d’un membre	✔	✔ (sauf propriétaire)	✘	✘	✘
Créer / archiver un projet	✔	✔	✘	✘	✘
Créer / modifier des colonnes	✔	✔	✔	✘	✘
Créer / modifier / déplacer des tâches	✔	✔	✔	✘	✘
Supprimer une tâche	✔	✔	Seulement ses tâches	✘	✘

Action	Propriétaire	Admin	Membre	Coach	Observateur
Écrire dans le chat	✔	✔	✔	✔	Lecture seule par défaut (option)
Déposer un fichier	✔	✔	✔	✔	✘
Supprimer un fichier	✔	✔	Seulement les siens	Seulement les siens	✘
Voir statistiques et rapports (M4)	✔	✔	✔	✔	✔
Exporter un rapport	✔	✔	✔	✔	✔
Créer / modifier une formation	✔	✔	✘	✔	✘
S’inscrire à une formation	✔	✔	✔	✘	✘
Demander une séance de coaching	✔	✔	✔	✘	✘
Répondre aux demandes de coaching, rédiger un compte rendu	✘	✘	✘	✔	✘
Voir les notes privées d’une séance	✘	✘	✘	Les siennes	✘
Créer ses objectifs de progression	✔	✔	✔	✘	✘
Évaluer les compétences d’un membre	✘	✘	✘	✔	✘

> **Règle d’or.** Toute vérification de permission est faite côté serveur, dans les endpoints Serverpod. Une application Flutter s’exécute sur la machine de l’utilisateur : son code peut être modifié ou contourné, et un client peut appeler le serveur sans passer par l’interface. L’interface masque les boutons interdits par confort, mais ne constitue jamais une protection. Cette règle s’applique en particulier aux **notes privées de coaching**, que seul le coach auteur peut lire.

5. Choix techniques

5.1 Pile technique

Domaine	Choix recommandé	Alternative
Framework client	Flutter (dernière version stable), Material 3	n/a (imposé)
Langage	Dart 3, analyse stricte (strict-casts, strict-raw-types)	n/a (imposé)
Serveur	Serverpod (endpoints, modèles, migrations, WebSocket, appels futurs)	Dart Frog + package postgres ; Supabase
Base de données	PostgreSQL en local (Docker Compose)	SQLite (non pris en charge par Serverpod)
Accès aux données	ORM de Serverpod (classes générées à partir des fichiers .spy.yaml)	Drift (si Dart Frog)
Authentification	Module d’authentification Serverpod : e-mail / mot de passe, Google / GitHub en option	Supabase Auth
Validation	Validateurs partagés (package studyflow_shared) côté client et serveur	formz
Gestion d’état	Riverpod (flutter_riverpod) avec AsyncNotifier	Bloc
Navigation	go_router (URL propres, liens profonds, paramètres de requête)	auto_route
Interface	Material 3, ThemeData et ThemeExtension, widgets adaptatifs	Fluent UI, shadcn_flutter
Glisser-déposer	Draggable / DragTarget natifs, ou appflowy_board	drag_and_drop_lists

Domaine	Choix recommandé	Alternative
Calendrier	table_calendar (MIT) ou syncfusion_flutter_calendar	Calendrier maison avec intl
Graphiques	fl_chart (statistiques du M4, radar de compétences du M7)	syncfusion_flutter_charts
Export PDF	Package pdf (Dart pur, donc aussi côté serveur) + printing pour l'aperçu	Génération côté client uniquement
Temps réel	Méthodes d'endpoint Serverpod renvoyant un Stream (WebSocket)	Server-Sent Events
Tâches planifiées	Appels futurs (FutureCall) de Serverpod	Script lancé à la main
Stockage de fichiers	Disque local (uploads/, hors de tout dossier servi), routes HTTP contrôlées	MinIO en local
E-mails de test	Mailpit (Docker Compose) + package mailer	Ethereal
Visioconférence	Lien externe saisi par le coach (Meet, Teams, Jitsi), aucune intégration	n/a
Sélection et dépôt de fichiers	file_picker, desktop_drop	cross_file
Rendu Markdown	markdown_widget (widgets, pas de HTML : pas d'injection de script)	markdown + rendu maison
Formulaires	Form, TextFormField + validateurs partagés	reactive_forms, flutter_form_builder
Dates et fuseaux	intl, timezone, flutter_timezone	jiffy
Internationalisation	flutter_localizations + fichiers ARB (français par défaut, anglais en option)	easy_localization

Domaine	Choix recommandé	Alternative
Stockage du jeton	flutter_secure_storage	shared_preferences (déconseillé)
Tests	flutter_test, mocktail, integration_test, outils de test Serverpod	patrol, golden tests
Qualité	dart format, dart analyze, flutter_lints ou very_good_analysis, lefthook	pre-commit
Version de Flutter	FVM (fichier .fvmrc versionné)	asdf
Intégration continue	GitHub Actions avec subosito/flutter-action	GitLab CI
Lancement	docker compose up -d, serveur Serverpod, flutter run -d chrome (tool/dev.sh)	Installation manuelle de PostgreSQL

5.2 Pourquoi un serveur Dart avec Serverpod

Flutter ne fournit que l'interface : un serveur séparé est nécessaire. Nous recommandons Serverpod pour trois raisons :

- Un seul langage pour les 7 développeurs, du widget à la requête SQL, et des modèles partagés : une classe Task est définie une fois et générée côté serveur et côté client.
- Tout ce dont le cahier des charges a besoin est déjà là : ORM relationnel avec migrations, authentification, flux temps réel par WebSocket, appels futurs planifiés, journalisation, cache local.
- Une exécution 100 % locale : seuls PostgreSQL et Mailpit tournent dans Docker.

Alternative : Supabase accélère le démarrage, mais déplace une partie de la logique dans des règles de sécurité SQL (Row Level Security) et n'est plus « tout en Dart ». Le choix est fixé en semaine 0 et ne change plus ensuite.

5.3 Points d'attention propres à Flutter

- **Premier chargement** : Flutter Web dessine l'interface sur un canevas, donc le chargement initial est plus lourd qu'une page HTML. La performance (§10.1) est exprimée en temps de démarrage et en fluidité.

- **Accessibilité sur le web** : l'arbre sémantique doit être activé et alimenté (Semantics, libellés, ordre de tabulation). C'est une tâche de chaque module, vérifiée en revue.
- **URL et liens profonds** : go_router est configuré avec des URL sans #, pour que les liens vers une tâche, un projet, une formation ou une invitation soient partageables.
- **Écosystème de composants** : les bibliothèques Kanban et calendrier sont moins mûres que leurs équivalents web. Les développeurs concernés (M2, M4) valident leur choix par un prototype d'une journée en semaine 1.
- **Densité du Module 4** : il regroupe calendrier et statistiques. Le Dev 4 démarre par le calendrier et les agrégations simples ; les graphiques avancés (burndown, charge) viennent en P2.
- **Code généré** : Serverpod génère du code versionné. Après chaque fusion, on exécute serverpod generate ; la CI échoue s'il reste une différence.
- **Montée en compétence** : prévoir un atelier Dart / Flutter / Riverpod de 2 jours en semaine 0 si l'équipe n'a pas d'expérience.
- **Temps réel** : l'application s'exécutant en local, le serveur Serverpod suffit : il expose les flux temps réel et exécute les tâches planifiées (rappels d'échéance, instantanés quotidiens, rappels de formation). Aucun service externe n'est nécessaire. Son architecture est fixée en semaine 1 par le Dev 5.

6. Architecture et conventions

6.1 Arborescence cible

```
studyflow/
├── docker-compose.yaml        # PostgreSQL + Mailpit
├── tool/dev.sh                # lance Docker, le serveur et l'application
├── .fvmrc                     # version de Flutter figée
├── .github/workflows/ci.yml
├── studyflow_shared/          # Dart pur : rôles, matrice de permissions,
│   └── lib/                   # validateurs, constantes (limites, tailles)
├── studyflow_server/          # Serveur Serverpod
│   ├── config/                # development.yaml, passwords.yaml (non
versionné)
│   ├── migrations/            # migrations SQL générées
│   ├── bin/                   # main.dart (serveur), seed.dart (données de
démo)
│   ├── test/
│   └── lib/src/
```

```
|       ├── models/            # un fichier .spy.yaml par entité (24)
|       ├── core/              # permissions.dart, position.dart,
activity.dart,
|       |                      # notify.dart, realtime.dart,
rate_limiter.dart,
|       |                      # calendar_source.dart
|       ├── modules/           # m1_auth_teams, m2_kanban, m3_tasks,
|       |                      # m4_calendar_stats, m5_chat, m6_files,
|       |                      # m7_training_coaching
|       ├── endpoints/         # un endpoint par module
|       ├── routes/            # upload, téléchargement, flux iCal (HTTP)
|       ├── future_calls/      # rappels d'échéance, rappels de formation,
|       |                      # instantanés quotidiens
|       └── generated/         # code généré (versionné, ne pas modifier)
├── studyflow_client/          # client Dart généré par Serverpod
├── studyflow_flutter/         # Application Flutter
|   ├── lib/
|   |   ├── main.dart, app.dart, router.dart
|   |   ├── core/              # thème, design system, l10n, session
|   |   └── features/          # auth_teams, kanban, tasks, calendar_stats,
|   |                          # chat, files, training_coaching
|   ├── l10n/                  # app_fr.arb, app_en.arb
|   ├── test/                  # tests unitaires et de widgets
|   └── integration_test/      # scénarios de bout en bout
├── uploads/                   # fichiers déposés (ignoré par Git)
└── README.md
```

Principe : chaque module vit dans un dossier `features/<nom>` côté Flutter et dans `modules/<nom>` côté serveur, et expose ses écrans au routeur. Chaque module contient `data/` (dépôts), `domain/` (logique pure) et `presentation/` (écrans). Cela limite les conflits Git.

6.2 Conventions de code

- **Nommage** : fichiers en snake_case, classes et widgets en PascalCase, variables et fonctions en camelCase. Entités en PascalCase singulier.
- **Langue** : code, commits et branches en anglais ; interface en français via les fichiers ARB (aucun texte en dur dans un widget).
- **Données** : toute entrée utilisateur est validée côté serveur avec les validateurs de studyflow_shared ; la validation côté client n'est qu'un confort.

- **Mutations** : via les méthodes d'endpoint, qui appellent assertPermission() (core/permissions.dart) avant toute écriture.
- **Erreurs** : le serveur lève des exceptions sérialisables (code + message) ; le client les convertit en un type scellé Result<T> = Ok(data) | Err(code, message).
- **Widgets** : préférer StatelessWidget et ConsumerWidget ; pas de logique métier dans build() ; widgets const dès que possible.
- **Accessibilité** : navigation clavier, libellés Semantics, contrastes AA, cibles tactiles d'au moins 48 px.
- **Analyse** : zéro avertissement de dart analyze ; formatage automatique par dart format.

6.3 Contrat entre modules

Pour que les 7 modules s'assemblent sans douleur, ces points sont figés dès la semaine 1 :

1. **Modèles de données** : chaque entité appartient à un module précis (§7), dans son propre fichier .spy.yaml. Toute modification passe par une pull request validée par le propriétaire de la table et par le Dev 3.
2. **core/permissions.dart (Dev 1)** : getCurrentUser(), requireTeamRole(session, teamId, minRole), requireProjectAccess(session, projectId), requireCoach(session, teamId). Tous les endpoints l'utilisent.
3. **Positionnement** des cartes et colonnes : champ position de type String avec fractional indexing (helper core/position.dart du Dev 3).
4. **Événements temps réel (Dev 5)** : convention objet.action, par exemple task.moved, message.created, file.uploaded, training.updated, coaching.confirmed. Canaux : team-{id}, project-{id} et task-{id}. API : publish(channel, event, payload) côté serveur, channelEventsProvider(channel) côté Flutter.
5. **Journal d'activité (M4, Dev 4)** : tout module appelle logActivity(session, projectId, userId, type, payload) après une mutation (core/activity.dart).
6. **Notifications (M5)** : tout module appelle notify(session, userId, type, payload) pour prévenir un utilisateur (assignation, mention, rappel, confirmation de séance).
7. **Sources du calendrier (M4)** : le M4 définit l'interface CalendarSource (core/calendar_source.dart). Le M4 fournit les échéances et les jalons ; le M7 fournit les formations et les séances de coaching. Le calendrier agrège toutes les sources sans connaître leur détail.
8. **Design system (Dev 2)** : widgets dans core/ui, couleurs et espacements dans le ThemeData et ses ThemeExtension. Aucune couleur écrite en dur.

7. Modèle de données

Les entités sont décrites dans le protocole Serverpod : un fichier .spy.yaml par entité, à partir duquel Serverpod génère les classes Dart, les tables PostgreSQL et les migrations. Les noms de tables SQL sont en snake_case.

7.1 Entités principales (24)

Le Module 4 porte 6 entités (3 de calendrier et 3 reprises de l'ancien M7), le Module 7 en porte 3 ; les autres modules en portent 3 chacun.

Module	Entité	Champs principaux
M1	User	id, name, email, image, createdAt (mot de passe géré par le module d'authentification Serverpod ; UserProfile en dépend)
M1	Team	id, name, description, slug, ownerId, inviteToken, inviteExpiresAt, createdAt
M1	TeamMember	id, teamId, userId, role (OWNER, ADMIN, MEMBER, COACH, VIEWER), joinedAt
M2	Project	id, teamId, name, description, color, startDate, endDate, archivedAt
M2	Column	id, projectId, name, position, wipLimit?, isDone
M2	ProjectTemplate	id, name, description, columns (JSON)
M3	Task	id, projectId, columnId, title, description, position, priority, dueDate, createdById, createdAt, completedAt, archivedAt
M3	Subtask	id, taskId, title, done, position
M3	Label	id, projectId, name, color
M4	Milestone	id, projectId, title, date, description
M4	Reminder	id, taskId, userId, remindAt, kind (H24, H1), sentAt?
M4	CalendarToken	id, userId, token, createdAt, revokedAt
M4	Activity	id, projectId, taskId?, userId, type, payload (JSON), createdAt

Module	Entité	Champs principaux
M4	ProjectSnapshot	id, projectId, date, total, done, overdue (instantané quotidien)
M4	Report	id, projectId, createdById, periodStart, periodEnd, data (JSON), createdAt
M5	Channel	id, teamId, projectId?, taskId?, name, type (TEAM, PROJECT, TASK)
M5	Message	id, channelId, authorId, content, replyToId?, createdAt, editedAt, deletedAt
M5	Notification	id, userId, type, payload (JSON), readAt, createdAt
M6	File	id, projectId, taskId?, folderId?, uploaderId, name, mimeType, size, currentVersion, createdAt
M6	Folder	id, projectId, parentId?, name
M6	FileVersion	id, fileId, version, storageKey, size, uploaderId, createdAt
M7	Training	id, teamId, title, description, skill, level (BEGINNER, INTERMEDIATE, ADVANCED), trainerId, startsAt, endsAt, capacity, mode (ONLINE, ONSITE), location, meetingUrl?, status (DRAFT, PUBLISHED, CANCELLED, DONE)
M7	CoachingSession	id, teamId, projectId?, coachId, coacheeId, topic, proposedSlots (JSON), scheduledAt?, durationMin, mode, meetingUrl?, status (REQUESTED, CONFIRMED, RESCHEDULED, DONE, CANCELLED), sharedReport?, privateNotes?, createdAt
M7	CoachingGoal	id, userId, teamId, coachId?, title, skill, targetDate, progress (0 à 100), status (OPEN, ACHIEVED, ABANDONED), createdAt

Tables de liaison et associées (non comptées dans les entités principales) :

- M3 : TaskAssignee (taskId, userId) et TaskLabel (taskId, labelId).
- M5 : MessageRead (channelId, userId, lastReadAt).

- M1 : tables du module d’authentification Serverpod (identifiants, sessions, jetons).
- M7 : TrainingEnrollment (trainingId, userId, status ENROLLED / WAITLIST / CANCELLED, attended, enrolledAt), TrainingResource (trainingId, title, url?, fileId?), Feedback (targetType TRAINING ou SESSION, targetId, authorId, rating 1 à 5, comment, createdAt) et SkillLevel (userId, teamId, skill, level 0 à 5, selfAssessed, assessedById?, updatedAt).

Les commentaires d’une tâche sont des Message d’un Channel de type TASK.

7.2 Relations clés

```
User ──< TeamMember >── Team ──< Project ──< Column ──< Task
 │                        │          │                    ├──< TaskAssignee
>── User
 │                        │          │                    ├──< Subtask
 │                        │          │                    ├──< TaskLabel >──
Label
 │                        │          │                    ├──< Reminder
 │                        │          │                    ├──1 Channel (TASK)
──< Message
 │                        │          │                    └──< File ──<
FileVersion
 │                        │          ├──< Label · Milestone
 │                        │          ├──< Folder ──< File
 │                        │          ├──< Channel (PROJECT) ──< Message
 │                        │          └──< Activity · ProjectSnapshot · Report
 │                        ├──< Channel (TEAM) ──< Message
 │                        ├──< Training ──< TrainingEnrollment >── User
 │                        │        ├──< TrainingResource
 │                        │        └──< Feedback
 │                        └──< CoachingSession (coach, coachee) ──< Feedback
 ├──< CoachingGoal · SkillLevel
 ├──< Notification
 └──< CalendarToken

ProjectTemplate : modèle indépendant, copié en colonnes à la création d’un
projet
```

7.3 Exemple de modèle Serverpod

```
# studyflow_server/lib/src/models/training.spy.yaml
class: Training
table: training
fields:
  team: Team?, relation(parent=team)
  title: String
  description: String?
  skill: String
  level: TrainingLevel     # enum : beginner, intermediate, advanced
  trainerId: int
  startsAt: DateTime
  endsAt: DateTime
  capacity: int
  mode: SessionMode        # enum : online, onsite
  location: String?
  meetingUrl: String?
  status: TrainingStatus   # enum : draft, published, cancelled, done
indexes:
  training_team_starts_idx:
    fields: team, startsAt
```

7.4 Règles d'intégrité

- Supprimer une équipe supprime en cascade ses projets, tâches, fichiers (y compris sur le disque), messages, formations, séances de coaching et objectifs.
- Un utilisateur ne peut être qu'une seule fois membre d'une même équipe (index unique sur teamId + userId) et une seule fois inscrit à une même formation (index unique sur trainingId + userId).
- Une équipe a toujours au moins un propriétaire.
- Une tâche appartient à exactement une colonne ; les colonnes d'un projet sont ordonnées par position.
- Les suppressions de messages et de tâches sont « douces » (deletedAt, archivedAt).
- Une formation passée n'est jamais supprimée, seulement marquée DONE ou CANCELLED, pour conserver les présences et les évaluations.
- Le nombre d'inscrits confirmés à une formation ne dépasse jamais sa capacité, y compris en cas d'inscriptions simultanées (transaction).

- Les notes privées d'une séance ne sont jamais renvoyées à un autre utilisateur que le coach auteur, y compris dans les exports et les flux temps réel.
- Index recommandés : Task(projectId, columnId, position), Task(dueDate), Message(channelId, createdAt), Activity(projectId, createdAt), Reminder(remindAt, sentAt), Training(teamId, startsAt), CoachingSession(coachId, scheduledAt).

8. Répartition de la charge de travail

8.1 Dimensionnement des modules

Le projet est découpé en 7 modules de charge identique, confiés chacun à un développeur. Les modules 4 et 7 sont redéfinis en v2.1 mais conservent la même charge : 12 fonctionnalités, 15 j-h, 1 responsabilité transverse de 3 j-h.

Critère	Valeur pour chaque module
Entités principales	3 (6 pour le M4, qui reprend celles de l'ancien M7)
Fonctionnalités	12 (6 en P1, 4 en P2, 2 en P3)
Charge des fonctionnalités	15 j-h (8 j-h en P1, 5 j-h en P2, 2 j-h en P3)
Responsabilité transverse	1, de 3 j-h
Charge totale par développeur	18 j-h
Écrans et panneaux principaux	5
Méthodes d'endpoint ou routes principales	8
Tests unitaires minimum	4
Défi technique	1

j-h = jour-homme : une journée de travail effectif d'une personne. Ces estimations sont revues une seule fois, à la fin de la semaine 1 (§8.4). Le M4 étant plus dense en entités, ses fonctionnalités sont volontairement formulées de façon groupée ; les graphiques avancés sont en P2.

8.2 Vue d’ensemble

Dev	Module	Entités principales	P1 / P2 / P3	Charge	Responsabilité transverse (3 j-h)	Total
Dev 1	M1 : Authentification et équipes	User, Team, TeamMember	6 / 4 / 2	15 j-h	Bibliothèque de permissions (dont requireCoach)	18 j-h
Dev 2	M2 : Tableaux Kanban	Project, Column, ProjectTemplate	6 / 4 / 2	15 j-h	Socle d’interface (coque, design system)	18 j-h
Dev 3	M3 : Tâches et assignations	Task, Subtask, Label	6 / 4 / 2	15 j-h	Schéma de données et migrations (24 modèles)	18 j-h
Dev 4	M4 : Calendrier, échéances, statistiques et rapports	Milestone, Reminder, CalendarToken, Activity, ProjectSnapshot, Report	6 / 4 / 2	15 j-h	Journal d’activité (logActivity)	18 j-h
Dev 5	M5 : Chat et notifications	Channel, Message, Notification	6 / 4 / 2	15 j-h	Temps réel et tâches planifiées (serveur)	18 j-h
Dev 6	M6 : Partage de fichiers	File, Folder, FileVersion	6 / 4 / 2	15 j-h	Intégration continue et sécurité	18 j-h
Dev 7	M7 : Formations et coaching	Training, CoachingSession, CoachingGoal	6 / 4 / 2	15 j-h	Tests de bout en bout	18 j-h
Total	7 modules	24 entités	42 / 28 / 14	105 j-h	7 responsabilités	126 j-h

Documentation et données de démonstration. Ces tâches, auparavant portées par le Dev 7, sont réparties : chaque développeur rédige la page docs/module-X.md de son module et un fragment de données de démo ; le Dev 7 assemble bin/seed.dart pendant ses tests de bout en bout, et le Dev 1 rédige le README. À confirmer en réunion de lancement.

8.3 Un défi technique par module

Module	Défi technique principal
M1	Sécuriser les sessions, les invitations et les droits par rôle (dont Coach)

Module	Défi technique principal
M2	Glisser-déposer fluide (60 images par seconde) avec un ordre persistant et des mises à jour optimistes
M3	Modèle de tâche riche, filtres combinés conservés dans l'URL (paramètres de requête go_router)
M4	Calendrier interactif multi-sources, fuseaux horaires, agrégations, burndown et génération du rapport PDF
M5	Messagerie temps réel par flux Dart, messages non lus et reprise après coupure
M6	Téléversement sécurisé avec progression, accès contrôlé et gestion des versions
M7	Inscriptions concurrentes sans dépassement de capacité, notes privées étanches, machine d'états des séances de coaching

8.4 Suivi de la charge pendant le projet

- Chaque fonctionnalité est une carte dans StudyFlow avec son estimation en j-h dans le titre (par exemple [1.5j]).
- Un point de suivi a lieu à la fin de chaque sprint : on compare les j-h réalisés et prévus de chaque développeur.
- Si l'écart entre deux développeurs dépasse 2 j-h, une fonctionnalité P2 ou P3 est transférée ou traitée en binôme. Les fonctionnalités P3 sont interchangeables entre modules.
- Les estimations sont revues une seule fois, à la fin de la semaine 1, avec les premières mesures réelles, puis figées. Tout changement ensuite est validé par toute l'équipe.
- Le bilan individuel du rapport final reprend, pour chaque fonctionnalité, les jours estimés et réalisés.

9. Spécifications fonctionnelles par module

Légende : P1 = indispensable (MVP), P2 = important, P3 = bonus. La charge est en j-h. Les « méthodes d'endpoint » désignent les méthodes publiques des endpoints Serverpod, appelées depuis Flutter par le client généré.

Module 1 : Authentification et gestion des équipes

Développeur : Dev 1 · **Entités** : User, Team, TeamMember (plus les tables d'authentification Serverpod) · **Charge** : 15 + 3 = 18 j-h

Objectif. Permettre à chacun de se connecter de façon sécurisée, de créer ou rejoindre des équipes, et de gérer les rôles de chaque membre, y compris le rôle Coach.

Réf.	Fonctionnalité	Priorité	Charge
1.1	Inscription par e-mail / mot de passe (validation, hachage fourni par Serverpod)	P1	1,5
1.2	Connexion / déconnexion, session persistante (jeton avec flutter_secure_storage)	P1	1
1.3	Protection des routes (redirection go_router vers /login) et limitation des tentatives de connexion	P1	1
1.4	Création d'une équipe et sélecteur d'équipe dans la barre de navigation (multi-équipes)	P1	1,5
1.5	Invitation par lien (jeton à durée limitée, révocation, **rôle proposé à l'invitation, dont Coach**) et écran d'acceptation (lien profond)	P1	1,5
1.6	Gestion des membres : liste, changement de rôle (dont Coach), retrait, départ volontaire	P1	1,5
1.7	Profil utilisateur : nom, photo, changement de mot de passe	P2	1,5
1.8	Connexion via Google et/ou GitHub (OAuth)	P2	1
1.9	Réinitialisation du mot de passe par e-mail (Mailpit en local)	P2	1,5
1.10	Paramètres d'équipe : renommer, transférer la propriété, supprimer l'équipe	P2	1

Réf.	Fonctionnalité	Priorité	Charge
1.11	Suppression du compte et export des données personnelles (RGPD)	P3	1
1.12	Invitation nominative par e-mail, en plus du lien	P3	1
	Total		**15**

Responsabilité transverse (3 j-h) : bibliothèque de permissions. Le Dev 1 écrit core/permissions.dart côté serveur (getCurrentUser(), requireTeamRole(session, teamId, minRole), requireProjectAccess(session, projectId), requireCoach(session, teamId), assertPermission()) et ses tests unitaires, qui reprennent la matrice du §4.2 (colonne Coach incluse). Il fournit aussi côté client un permissionsProvider Riverpod qui sert uniquement à masquer les boutons interdits. Livrée en semaine 1.

Règles de gestion

- Mot de passe : 8 caractères minimum, avec vérification de force.
- Un lien d'invitation expire au bout de 7 jours et peut être révoqué ou régénéré.
- Un propriétaire ne peut pas quitter l'équipe sans transférer la propriété.
- Un e-mail ne peut être associé qu'à un seul compte.
- Limitation du nombre de tentatives de connexion (anti brute-force).
- Seuls le propriétaire et les administrateurs peuvent attribuer le rôle Coach.

Écrans (5) : authentification (/login, /register, /forgot-password, /reset-password) · profil (/me) · mes équipes et création (/teams, /teams/new) · paramètres d'équipe : membres, rôles, lien d'invitation (/teams/:teamId/settings) · acceptation d'une invitation (/invite/:token).

Méthodes d'endpoint (8) : register, updateProfile, createTeam, manageTeam (renommer, transférer, supprimer), regenerateInviteLink, acceptInvitation, updateMemberRole, removeMember.

Tests unitaires (4 minimum) : validation et force du mot de passe ; expiration d'un lien d'invitation ; transfert de propriété ; rôle minimal requis par action (dont les actions réservées au Coach).

Critères d'acceptation

- Un utilisateur non connecté qui ouvre un écran protégé est redirigé vers /login, puis ramené sur sa page après connexion.

- Un membre simple ne peut ni inviter ni retirer quelqu'un, y compris en appelant directement la méthode d'endpoint.
- Un lien d'invitation expiré affiche un message clair.
- Un utilisateur invité avec le rôle Coach ne peut pas modifier une tâche, même par appel direct au serveur.

Dépendances : aucune en entrée. Tous les autres modules dépendent de ses permissions.

Module 2 : Tableaux Kanban (glisser-déposer)

Développeur : Dev 2 · **Entités** : Project, Column, ProjectTemplate · **Charge** : 15 + 3 = 18 j-h

Objectif. Offrir une vue visuelle et interactive du travail de l'équipe, organisée en colonnes représentant les étapes d'avancement.

Réf.	Fonctionnalité	Priorité	Charge
2.1	Création et modification d'un projet (nom, description, couleur, dates)	P1	1
2.2	Liste des projets de l'équipe, archivage et restauration	P1	1
2.3	Colonnes : 4 par défaut (À faire, En cours, En relecture, Terminé), ajout, renommage, suppression	P1	1,5
2.4	Réordonnancement des colonnes par glisser-déposer	P1	1
2.5	Déplacement des cartes par glisser-déposer, entre colonnes et dans une colonne, position persistante	P1	3
2.6	Création rapide d'une carte depuis le bas d'une colonne	P1	0,5
2.7	Carte résumée : titre, étiquettes, avatars des assignés, échéance, compteurs de sous-tâches et de pièces jointes	P2	1,5
2.8	Mises à jour optimistes (la carte bouge immédiatement, retour arrière en cas d'erreur)	P2	1

Réf.	Fonctionnalité	Priorité	Charge
2.9	Filtres du tableau (assigné, étiquette, priorité, échéance) et recherche texte	P2	1,5
2.10	Accessibilité du glisser-déposer : clavier, lecteur d'écran (Semantics) et écran tactile	P2	1
2.11	Création d'un projet à partir d'un modèle (Sprint de 2 semaines, Projet de fin d'année)	P3	1
2.12	Limite de cartes par colonne (WIP) avec avertissement visuel, et couleur de colonne	P3	1
	Total		**15**

Responsabilité transverse (3 j-h) : socle d'interface. Le Dev 2 réalise la coque (AppShell : barre latérale ou NavigationRail sur grand écran, barre basse sur mobile), le design system partagé (core/ui : ThemeData Material 3, ThemeExtension pour couleurs et espacements) et les widgets de chargement (squelettes), d'erreur et d'état vide. Livré en semaine 1.

Règles de gestion

- Déplacer une carte dans la colonne marquée isDone renseigne completedAt ; la sortir de cette colonne le remet à null.
- Supprimer une colonne non vide demande de choisir où déplacer ses cartes (ou de les supprimer explicitement).
- Un nouveau projet reçoit les 4 colonnes par défaut ou celles du modèle choisi.
- La limite WIP avertit sans bloquer.
- Seuls les rôles Membre et supérieurs peuvent déplacer des cartes (le Coach et l'Observateur sont en lecture).
- Chaque déplacement alimente le journal d'activité (task.moved, avec colonne d'origine et de destination) et publie l'événement temps réel correspondant.

Écrans (5) : liste des projets et archives (/teams/:teamId/projects) · formulaire de projet · tableau Kanban (/teams/:teamId/projects/:projectId/board) · panneau de gestion des colonnes · barre de filtres.

Méthodes d'endpoint (8) : createProject, updateProject (modifier, archiver, restaurer), listProjects, createColumn, updateColumn, deleteColumn, reorderColumns, moveTask.

Tests unitaires (4 minimum) : calcul des positions (insertion entre deux cartes) ; passage dans la colonne terminée et completedAt ; suppression d'une colonne non vide ; application des filtres.

Critères d'acceptation

- Déplacer une carte entre deux colonnes puis relancer l'application conserve la position exacte.
- Deux utilisateurs déplaçant des cartes différentes en même temps ne se gênent pas ; en cas de conflit sur la même carte, la dernière écriture gagne et l'interface se resynchronise.
- Le tableau reste fluide (60 images par seconde) avec 100 cartes réparties sur 6 colonnes.

Dépendances : M1 (permissions), M3 (modèle Task), M5 (synchronisation en direct), M4 (logActivity).

Module 3 : Gestion des tâches et assignations

Développeur : Dev 3 · **Entités** : Task, Subtask, Label (plus TaskAssignee et TaskLabel) · **Charge** : 15 + 3 = 18 j-h

Objectif. Fournir le cœur « métier » de l'application : une tâche riche, assignable, avec sa propre vue détaillée.

Réf.	Fonctionnalité	Priorité	Charge
3.1	Création et modification d'une tâche : titre, description (Markdown), priorité (basse, normale, haute, urgente), échéance	P1	1,5
3.2	Fenêtre de détail d'une tâche (feuille modale ou panneau latéral, lien profond partageable)	P1	2
3.3	Assignation à un ou plusieurs membres de l'équipe	P1	1,5
3.4	Étiquettes du projet (nom + couleur) et affectation aux tâches	P1	1,5
3.5	Sous-tâches (checklist) avec barre de progression	P1	1
3.6	Suppression, archivage et duplication d'une tâche	P1	0,5

Réf.	Fonctionnalité	Priorité	Charge
3.7	Vue liste / tableau des tâches du projet : tri, filtres, recherche, filtres conservés dans l'URL	P2	2
3.8	Fil de commentaires dans le détail de la tâche (widget de discussion de M5)	P2	1
3.9	Historique de la tâche : qui a changé quoi, quand (lecture du journal d'activité du **M4**)	P2	1
3.10	Règles métier et validations (assignation limitée aux membres, longueurs, tâche en retard) avec tests	P2	1
3.11	Actions groupées (assigner, changer la priorité, déplacer plusieurs tâches)	P3	1
3.12	Dépendances entre tâches (« bloquée par »)	P3	1
	Total		**15**

Responsabilité transverse (3 j-h) : schéma de données. Le Dev 3 écrit les **24 modèles** du protocole Serverpod (un .spy.yaml par entité) dès la semaine 1, gère les migrations (serverpod create-migration), fournit core/position.dart (fractional indexing) et le type de résultat standard côté client. Il relit toutes les pull requests qui touchent un .spy.yaml ou une migration.

Règles de gestion

- Seuls les membres de l'équipe peuvent être assignés à une tâche du projet (ni le Coach ni l'Observateur).
- Une tâche en retard (échéance dépassée et non terminée) est signalée visuellement.
- Titre obligatoire (1 à 120 caractères) ; description limitée à 10 000 caractères.
- La suppression d'une tâche supprime ses sous-tâches et son fil de discussion. Ses fichiers sont conservés dans le M6 avec taskId = null, ou supprimés selon le choix de l'utilisateur.
- Une assignation déclenche une notification (notify() de M5) et une entrée dans le journal d'activité.

Écrans (5) : liste / tableau des tâches (/teams/:teamId/projects/:projectId/tasks) · détail d'une tâche (…/tasks/:taskId, aussi en feuille modale depuis le Kanban) · formulaire de création et d'édition · gestionnaire d'étiquettes · panneau des sous-tâches.

Méthodes d'endpoint (8) : createTask, updateTask, deleteTask, duplicateTask, setAssignees, saveLabel, saveSubtask, listTasks.

Tests unitaires (4 minimum) : assignation limitée aux membres ; validation des longueurs ; progression des sous-tâches ; combinaison de filtres.

Critères d'acceptation

- Assigner une tâche à un utilisateur la fait apparaître dans sa page « Mes tâches » (M4) et déclenche une notification.
- Cocher toutes les sous-tâches ne termine pas automatiquement la tâche, mais le propose.
- Les filtres de la vue liste sont conservés dans l'URL et donc partageables.

Dépendances : M1 (membres, permissions), M2 (colonnes), M4 (journal d'activité), M5 (discussion, notifications), M6 (pièces jointes).

Module 4 : Calendrier, échéances, statistiques et rapports

Développeur : Dev 4 · **Entités** : Milestone, Reminder, CalendarToken, Activity, ProjectSnapshot, Report · **Charge** : 15 + 3 = 18 j-h

Objectif. Offrir une vision temporelle du projet pour anticiper les deadlines, équilibrer la charge de travail, et transformer les données du projet en indicateurs lisibles (avancement, blocages, rapports) pour les points d'étape et la soutenance. Le M4 est aussi le point d'agrégation du calendrier : il consomme les sources fournies par les autres modules (dont les formations et séances du M7).

Réf.	Fonctionnalité	Priorité	Charge
4.1	Calendrier mensuel agrégeant toutes les sources (CalendarSource) : échéances de tâches, jalons, formations et séances de coaching, avec code couleur par type	P1	2
4.2	Calendrier par projet et calendrier personnel (tous mes projets) ; appui sur un événement : détail de la tâche (M3), de la formation ou de la séance (M7) ; mise en évidence des retards et des échéances du jour	P1	1

Réf.	Fonctionnalité	Priorité	Charge
4.3	Jalons (milestones) : dates clés du projet (rendu, soutenance, livraison), création, modification, suppression	P1	1
4.4	Écran « Mes tâches » : toutes mes tâches, tous projets confondus, groupées par échéance	P1	1
4.5	Fil d'activité « qui a fait quoi » du projet (lecture de la table Activity) et sélecteur de période (7 jours, 30 jours, depuis le début)	P1	1,5
4.6	Tableau de bord du projet : % d'avancement, tâches totales / terminées / en retard, répartition par colonne et par priorité (graphiques fl_chart)	P1	1,5
4.7	Vues semaine et liste (agenda), modification de l'échéance par glisser-déposer d'une tâche vers un autre jour	P2	1,5
4.8	Rappels automatiques 24 h et 1 h avant l'échéance (appel futur Serverpod)	P2	1
4.9	Charge par membre (assignées, terminées, en retard), instantané quotidien, burndown et vélocité	P2	1,5
4.10	Rapport d'avancement : aperçu à l'écran, enregistrement en base, export PDF (package pdf)	P2	1
4.11	Export iCal avec lien d'abonnement personnel secret, et export CSV des données	P3	1
4.12	Alertes (tâche bloquée depuis plus de X jours, projet en retard, membre inactif) et tableau de bord global de l'utilisateur (/dashboard)	P3	1
	Total		**15**

Responsabilité transverse (3 j-h) : journal d'activité. Le Dev 4 écrit core/activity.dart et la fonction logActivity(session, projectId, userId, type, payload), appelée par tous les modules après une mutation, ainsi que le catalogue des types d'activité (task.created, task.moved, task.assigned, file.uploaded, training.created, coaching.confirmed, etc.). Il fournit aussi l'interface CalendarSource (core/calendar_source.dart). Livré en semaine 1, avec des données fictives pour les modules qui démarrent.

Règles de gestion

- Les tâches sans échéance n'apparaissent pas dans le calendrier : elles sont listées dans un panneau « Non planifiées ».
- Les jalons sont créés par les administrateurs et propriétaires uniquement.
- Les dates sont stockées en UTC et affichées dans le fuseau horaire de l'utilisateur (timezone, flutter_timezone). Le premier jour de la semaine est le lundi.
- Un rappel est envoyé une seule fois (sentAt) via notify(). Le lien d'abonnement iCal est secret et peut être régénéré.
- L'avancement global est la proportion de tâches dans la colonne « terminé ». Une tâche est « en retard » si dueDate < aujourd'hui et completedAt est vide.
- Les statistiques de charge par membre sont visibles de tous, mais ne servent pas à classer ou sanctionner. Une option permet de les masquer.
- Les calculs lourds sont faits côté serveur (requêtes SQL agrégées) et mis en cache (cache local de Serverpod, avec durée de vie).
- Un instantané du projet est enregistré chaque nuit par un appel futur ; il alimente le burndown.
- Le rapport est daté et reproductible (date et période de génération). Tous les rôles, Coach et Observateur compris, peuvent consulter et exporter statistiques et rapports.
- Le calendrier n'affiche que les formations et séances que l'utilisateur a le droit de voir (voir M7) ; la séance d'un autre membre n'apparaît jamais.

Écrans (5) : calendrier du projet, mois / semaine / liste (/teams/:teamId/projects/:projectId/calendar) · calendrier personnel (/me/calendar) et Mes tâches (/me/tasks) · jalons et abonnement iCal · tableau de bord et journal d'activité (/teams/:teamId/projects/:projectId/stats) · aperçu et historique des rapports (…/stats/report).

Méthodes d'endpoint principales (8) : listCalendarEvents, listMyTasks, saveMilestone, getProjectStats, getBurndown, listActivity, generateReport, exportReportPdf.

Méthodes complémentaires : deleteMilestone, updateDueDate, getMemberWorkload, getVelocity, exportCsv, regenerateCalendarToken, route HTTP GET /calendar/<token>.ics, appels futurs sendDueReminders et takeDailySnapshot.

Tests unitaires (4 minimum) : construction des événements d'un mois à partir de plusieurs sources ; conversion de fuseaux horaires ; sélection des rappels à envoyer ; calcul de l'avancement et détection des retards ; calcul du burndown ; génération du fichier iCal.

Critères d’acceptation

- Modifier l’échéance d’une tâche dans le détail met à jour le calendrier sans rechargement manuel.
- Une formation publiée ou une séance confirmée apparaît dans le calendrier de ses participants sans modification du code du M4 (simple ajout d’une source).
- Un jalon créé est visible par tous les membres du projet.
- Déplacer une tâche en « Terminé » met à jour le pourcentage d’avancement à la prochaine consultation (ou en direct si le temps réel est branché).
- Le rapport PDF contient au minimum : titre du projet, période, avancement, graphique de répartition, charge par membre, tâches en retard.
- Calendrier et graphiques sont lisibles et utilisables sur un écran de 375 px, avec légendes et couleurs accessibles.

Dépendances : M1 (identité, fuseau, membres), M2 et M3 (tâches, colonnes, échéances), M5 (notify()), M7 (sources formations et séances), appels futurs Serverpod (mis en place par le Dev 5).

Module 5 : Chat en temps réel et notifications

Développeur : Dev 5 · **Entités** : Channel, Message, Notification (plus MessageRead) · **Charge** : 15 + 3 = 18 j-h

Objectif. Faciliter la communication de l’équipe et tenir chacun informé en direct de ce qui le concerne.

Réf.	Fonctionnalité	Priorité	Charge
5.1	Salon général par équipe et salon par projet, créés automatiquement	P1	1
5.2	Envoi et réception de messages en temps réel	P1	1,5
5.3	Historique persistant avec chargement progressif (pagination vers le haut)	P1	1,5
5.4	Centre de notifications : cloche, liste, marquage comme lu	P1	1,5
5.5	Génération des notifications (assignation, commentaire, mention, rappel, **confirmation de séance, inscription à une formation**) via notify()	P1	1

Réf.	Fonctionnalité	Priorité	Charge
5.6	Synchronisation en direct du Kanban et des tâches (événements task.*)	P1	1,5
5.7	Fil de discussion d'une tâche (canal TASK), widget réutilisé par M3	P2	1,5
5.8	Messages non lus : badge par salon et marqueur « nouveaux messages »	P2	1
5.9	Mentions @membre et références de tâche #42 (notification et lien)	P2	1,5
5.10	Modification et suppression de ses propres messages, Markdown simple	P2	1
5.11	Indicateur « en train d'écrire » et présence en ligne	P3	1
5.12	Réactions par émojis et réponse à un message	P3	1
	Total		**15**

Responsabilité transverse (3 j-h) : temps réel et tâches planifiées. Le Dev 5 met en place le canal temps réel (méthodes d'endpoint renvoyant un Stream, WebSocket) avec contrôle des abonnements, l'API publish(channel, event, payload) et le provider channelEventsProvider(channel) côté Flutter, ainsi que la configuration des appels futurs (FutureCall) utilisés par M4 (rappels d'échéance, instantanés quotidiens) et M7 (rappels de formation et de séance). Opérationnel à la fin de la semaine 1.

Règles de gestion

- Seuls les membres d'une équipe (Coach compris) peuvent lire et écrire dans ses salons.
- Un message est limité à 4 000 caractères ; l'envoi d'un message vide est interdit.
- Un message supprimé est remplacé par « Message supprimé » (suppression douce).
- Limitation de débit : 20 messages par minute et par utilisateur.
- L'abonnement à un canal est vérifié côté serveur.
- Le canal d'une tâche est créé au premier message.
- Les événements training.updated et coaching.confirmed sont publiés sur le canal team-{id} ; le contenu des comptes rendus et notes privées n'y transite jamais.

Écrans (5) : salon d'équipe (/teams/:teamId/chat) · salon de projet (/teams/:teamId/projects/:projectId/chat) · fil de discussion d'une tâche (intégré au détail) · centre de notifications (/me/notifications) · panneau de chat rétractable.

Méthodes d'endpoint (8) : sendMessage, editMessage, deleteMessage, listMessages (par curseur), markChannelRead, getTaskChannel, listNotifications, markNotificationRead (plus notify() et le flux watchChannel).

Tests unitaires (4 minimum) : limitation de débit ; analyse des mentions et des références #42 ; pagination par curseur ; calcul des messages non lus.

Critères d'acceptation

- Un message apparaît chez les autres en moins de 1 seconde, sans rechargement.
- Après une coupure de connexion, la reconnexion récupère les messages manqués.
- Un non-membre qui tente de lire un salon reçoit une erreur d'accès refusé (exception sérialisable de type 403).
- L'assignation d'une tâche et la confirmation d'une séance de coaching créent une notification visible dans la cloche de la personne concernée.

Dépendances : M1 (identité, appartenance), M3 (références de tâches), M4 (journal), M7 (événements formations et coaching), serveur Serverpod (temps réel).

Module 6 : Partage de fichiers

Développeur : Dev 6 · **Entités** : File, Folder, FileVersion · **Charge** : 15 + 3 = 18 j-h

Objectif. Centraliser les documents du projet (cahiers des charges, maquettes, comptes rendus, supports de formation, rendus finaux) et les relier aux tâches.

Réf.	Fonctionnalité	Priorité	Charge
6.1	Téléversement par bouton ou glisser-déposer (file_picker, desktop_drop), avec progression et contrôles (taille, type)	P1	2
6.2	Liste des fichiers du projet (nom, type, taille, auteur, date) avec tri et recherche	P1	1
6.3	Téléchargement contrôlé (accès par session ou lien signé à durée limitée)	P1	1

Réf.	Fonctionnalité	Priorité	Charge
6.4	Pièces jointes liées à une tâche (widget dans le détail de la tâche et sur la carte)	P1	1,5
6.5	Renommage et suppression (selon permissions, avec effacement sur le disque)	P1	1
6.6	Dossiers et sous-dossiers, déplacement de fichiers	P1	1,5
6.7	Aperçu intégré : images, PDF, texte / Markdown	P2	1,5
6.8	Quota de stockage par équipe avec jauge d'utilisation	P2	1
6.9	Versionnage : remplacer un fichier en gardant l'historique	P2	1,5
6.10	Vignettes pour les images	P2	1
6.11	Lien de partage temporaire, désactivable	P3	1
6.12	Téléchargement d'un dossier en archive ZIP	P3	1
	Total		**15**

Responsabilité transverse (3 j-h) : intégration continue et sécurité. Le Dev 6 configure GitHub Actions (dart analyze, dart format, tests unitaires et de widgets, vérification que le code généré est à jour), fournit l'outil de limitation de débit utilisé par M1 et M5, ajoute les contrôles de sécurité du serveur (en-têtes, CORS restreint) et mène l'audit des accès en semaine 6, notes privées de coaching comprises.

Règles de gestion

- Taille maximale par fichier : 20 Mo (configurable). Quota par équipe : 500 Mo (configurable).
- Types autorisés : documents (pdf, docx, xlsx, pptx, txt, md), images (png, jpg, gif, webp, svg), archives (zip), code source courant. Les exécutables sont refusés.
- Le type MIME est vérifié côté serveur (signature du fichier), pas seulement l'extension.
- Jamais d'URL publique permanente : accès par session ou lien signé.
- Un fichier supprimé est effacé du disque, pas seulement de la base.
- Les noms de fichiers sont nettoyés (caractères dangereux, chemins relatifs).
- Les 10 dernières versions d'un fichier sont conservées.

- Un fichier peut servir de ressource à une formation (TrainingResource.fileId) : il reste soumis aux droits de l'équipe.

Écrans (5) : explorateur de fichiers (/teams/:teamId/projects/:projectId/files) · fenêtre d'aperçu · zone de dépôt et de progression · pièces jointes d'une tâche (widget exposé à M3) · quota et historique des versions.

Méthodes d'endpoint (8) : route HTTP POST /upload, listFiles, getDownloadUrl, renameFile, moveFile, deleteFile, saveFolder, replaceFile.

Tests unitaires (4 minimum) : validation du type et de la taille ; nettoyage du nom de fichier ; calcul du quota ; contrôle d'accès au téléchargement.

Critères d'acceptation

- Un non-membre de l'équipe ne peut pas télécharger un fichier, même en connaissant son identifiant.
- Un fichier de 15 Mo est téléversé avec progression et apparaît dans la liste sans rechargement.
- Supprimer un fichier retire la pièce jointe de la tâche associée.
- Remplacer un fichier garde l'ancienne version téléchargeable.

Dépendances : M1 (permissions), M3 (pièces jointes), M5 (événement file.uploaded, optionnel), M7 (ressources de formation).

Module 7 : Formations et coaching

Développeur : Dev 7 · **Entités** : Training, CoachingSession, CoachingGoal (plus TrainingEnrollment, TrainingResource, Feedback, SkillLevel) · **Charge** : 15 + 3 = 18 j-h

Objectif. Organiser la montée en compétences de l'équipe : proposer des formations internes, permettre aux membres de s'y inscrire, de demander des séances de coaching individuelles, de fixer des objectifs et de suivre l'évolution de leurs compétences, le tout relié au calendrier et aux notifications.

Réf.	Fonctionnalité	Priorité	Charge
7.1	Catalogue des formations : liste, filtres (compétence, niveau, mode, statut), page de détail	P1	1,5
7.2	Création, modification, publication et annulation d'une formation (administrateur ou coach)	P1	1,5

Réf.	Fonctionnalité	Priorité	Charge
7.3	Inscription et désinscription, avec capacité, liste d'attente et promotion automatique (transaction)	P1	1,5
7.4	Demande de séance de coaching : sujet, projet lié, jusqu'à 3 créneaux proposés	P1	1,5
7.5	Réponse du coach : confirmer un créneau, reprogrammer ou refuser, avec notification	P1	1
7.6	Intégration au calendrier (M4) : sources TrainingCalendarSource et CoachingCalendarSource, et notifications associées	P1	1
7.7	Présence : émargement par le formateur pendant ou après la formation	P2	1
7.8	Compte rendu de séance : partie partagée avec le coaché, notes privées du coach, liste d'actions	P2	1,5
7.9	Objectifs de progression (CoachingGoal) : création, avancement 0 à 100, statut	P2	1
7.10	Compétences (SkillLevel) : auto-évaluation par le membre, évaluation par le coach, radar fl_chart sur /me/progress	P2	1,5
7.11	Évaluation (Feedback, note de 1 à 5 et commentaire) des formations et des séances terminées	P3	1
7.12	Ressources de formation (liens ou fichiers du M6) et rappels automatiques avant une formation ou une séance (appel futur)	P3	1
	Total		**15**

Responsabilité transverse (3 j-h) : tests de bout en bout. Le Dev 7 met en place les tests d'intégration Flutter (integration_test) avec un serveur Serverpod de test et écrit les **7 scénarios** de bout en bout, qui servent aussi de recette : inscription, création d'équipe, création et déplacement d'une tâche, envoi d'un message, dépôt d'un fichier, inscription à une formation, demande et confirmation d'une séance de coaching. Il assemble aussi bin/seed.dart (§8.2).

Règles de gestion

- Seuls les formations au statut PUBLISHED sont visibles des membres ; les brouillons (DRAFT) ne sont visibles que de leur auteur et des administrateurs.
- Une formation passée n'est jamais supprimée : elle passe en DONE ou CANCELLED, pour conserver présences et évaluations.
- Le nombre d'inscrits confirmés ne dépasse jamais la capacité, y compris en cas d'inscriptions simultanées (transaction). Au-delà, l'inscription passe en WAITLIST ; une désinscription promeut le premier de la liste et le notifie.
- Une personne ne peut s'inscrire qu'une fois à une même formation. Le Coach ne s'inscrit pas aux formations (il les anime).
- Une séance suit les états REQUESTED → CONFIRMED → DONE, avec les branches RESCHEDULED (retour à CONFIRMED) et CANCELLED. Toute transition invalide est refusée par le serveur.
- Seul le coach désigné peut répondre à une demande et rédiger le compte rendu ; le coaché demande, annule ou reprogramme sa demande.
- **Étanchéité des notes privées** : privateNotes n'est lu que par le coach auteur, côté serveur, et exclu de toute réponse destinée à un autre utilisateur, de tout export, rapport, flux temps réel ou notification. Le sharedReport n'est lisible que par le coach et le coaché.
- Les administrateurs voient la liste des séances de l'équipe (sujet, statut, date) mais pas les comptes rendus.
- Les séances en ligne utilisent un lien externe saisi par le coach (meetingUrl) ; aucune visioconférence n'est intégrée.
- Évaluer les compétences d'un membre est réservé au Coach ; un membre peut renseigner sa propre auto-évaluation (selfAssessed = true).
- Chaque action alimente le journal d'activité (training.created, training.enrolled, coaching.requested, coaching.confirmed) et publie l'événement temps réel correspondant.

Écrans (5) : catalogue des formations (/teams/:teamId/trainings) · détail d'une formation, inscrits et présence (/teams/:teamId/trainings/:trainingId) · séances de coaching, demandes et historique (/teams/:teamId/coaching) · détail d'une séance et compte rendu (/teams/:teamId/coaching/:sessionId) · progression personnelle : objectifs et radar de compétences (/me/progress).

Méthodes d'endpoint principales (8) : listTrainings, saveTraining, enrollInTraining (et annulation), markAttendance, requestCoaching, respondToCoaching, saveSessionReport, saveGoal.

Méthodes complémentaires : listMySessions, saveSkillLevel, listSkillLevels, submitFeedback, saveTrainingResource, appel futur sendTrainingReminders.

Tests unitaires (4 minimum) : capacité, liste d'attente et promotion ; transitions d'état d'une séance ; permission de lecture des notes privées (seul le coach auteur) ; calcul de l'avancement d'un objectif et des niveaux de compétence.

Critères d'acceptation

- Deux inscriptions simultanées à la dernière place ne dépassent jamais la capacité : l'une est confirmée, l'autre mise en liste d'attente.
- Un membre peut s'inscrire à une formation et demander une séance en moins de 2 minutes.
- Un utilisateur autre que le coach auteur ne peut pas lire les notes privées, y compris en appelant directement l'endpoint.
- La confirmation d'une séance notifie le coaché et ajoute la séance au calendrier des deux personnes.
- Le radar de compétences est lisible sur un écran de 375 px.

Dépendances : M1 (rôle Coach, requireCoach), M4 (CalendarSource, journal d'activité), M5 (notifications, temps réel), M6 (ressources, optionnel), appels futurs Serverpod (mis en place par le Dev 5).

10. Exigences non fonctionnelles

10.1 Performance

- Démarrage : premier écran utilisable en moins de 4 secondes en local sur Flutter Web (build release, navigateur récent), mesuré et consigné dans le rapport de tests.
- Fluidité : défilement et glisser-déposer à 60 images par seconde avec 100 cartes (Flutter DevTools, mode profile).
- Interactions (déplacement de carte, envoi de message, inscription à une formation) : retour visuel immédiat (moins de 100 ms perçu, grâce aux mises à jour optimistes).
- Requêtes paginées pour toute liste susceptible de dépasser 50 éléments ; listes affichées avec ListView.builder.

- Pas de requêtes N+1 (relations incluses de l'ORM Serverpod). Agrégations du M4 faites en SQL et mises en cache.
- Taille de l'application web : surveiller le poids du build release et éviter les dépendances inutiles.

10.2 Sécurité

- Mots de passe hachés par le module d'authentification Serverpod, jamais journalisés.
- Jeton de session conservé avec flutter_secure_storage ; sessions révocables.
- Contrôle d'accès vérifié côté serveur pour chaque lecture et écriture (équipe, projet, ressource, séance de coaching) : le client n'est jamais considéré comme fiable.
- Notes privées de coaching : lisibles uniquement par le coach auteur, exclues des réponses, exports, flux temps réel et notifications ; couvertes par des tests dédiés et par l'audit de la semaine 6.
- Contenu utilisateur affiché comme texte ou Markdown rendu en widgets (aucun HTML brut) ; liens sortants contrôlés (dont meetingUrl : schéma https uniquement).
- CORS limité à l'adresse de l'application ; en-têtes de sécurité sur les routes HTTP.
- Secrets uniquement dans config/passwords.yaml et variables d'environnement, jamais dans Git ; fournir passwords.yaml.example.
- Limitation de débit sur les routes sensibles (connexion, inscription, messages, upload, inscriptions aux formations).
- Dépendances pub surveillées (Dependabot) et vérifiées avec dart pub outdated.

10.3 Données personnelles (RGPD)

- Collecte minimale (nom, e-mail, photo facultative). Les évaluations de compétences et comptes rendus de coaching sont des données personnelles : ils sont inclus dans l'export et la suppression du compte (fonctionnalité 1.11).
- Possibilité de supprimer son compte et d'exporter ses données.
- Écran de politique de confidentialité et mentions légales.
- Aucun traceur tiers ni outil d'analyse sans consentement.

10.4 Accessibilité et ergonomie

- Conformité visée : WCAG 2.1 niveau AA sur les parcours principaux.
- Navigation complète au clavier (y compris le déplacement de cartes avec Shortcuts et Actions), focus visible, libellés Semantics, arbre sémantique activé sur Flutter Web.

- Contraste suffisant, information jamais transmise par la seule couleur (important pour le calendrier, les graphiques et le radar de compétences) ; texte compatible avec l'agrandissement des polices.
- Interface adaptative : ordinateur (≥ 1024 px), tablette, mobile (≥ 375 px), avec des points de rupture communs définis par le design system.
- États de chargement (squelettes), états vides pédagogiques et messages d'erreur compréhensibles.

10.5 Qualité et maintenabilité

- Zéro avertissement de dart analyze, formatage par dart format, hooks Git (lefthook).
- Couverture de tests : au minimum 4 tests unitaires par module sur la logique critique, des tests de widgets sur les écrans clés, et **7 scénarios de bout en bout** (inscription, création d'équipe, création et déplacement d'une tâche, envoi d'un message, dépôt d'un fichier, inscription à une formation, demande et confirmation d'une séance de coaching).
- README à jour (installation en moins de 10 minutes) et une page docs/module-X.md par module.

10.6 Compatibilité

- Navigateurs : dernières versions de Chrome, Firefox, Safari et Edge (Chrome pour la démonstration).
- Flutter et Dart : même version pour toute l'équipe, figée avec FVM ; une seule version de Dart pour le serveur et le client.
- Docker (ou Docker Desktop) pour lancer PostgreSQL et Mailpit en local.

10.7 Exécution en local

Le lancement se fait en trois commandes, regroupées dans tool/dev.sh :

```
docker compose up -d   # PostgreSQL + Mailpit
cd studyflow_server && dart bin/main.dart --apply-migrations   # serveur
Serverpod
cd studyflow_flutter && flutter run -d chrome --dart-
define=API_URL=http://localhost:8080/
```

Jeu de données de démonstration (bin/seed.dart) : 1 équipe, 5 membres dont 1 coach, 2 projets, environ 40 tâches, messages et fichiers d'exemple, 3 formations (dont une complète avec liste d'attente), 4 séances de coaching à des états différents, des objectifs et des niveaux de compétence. tool/db_reset.sh puis dart bin/seed.dart le recréent à l'identique avant chaque démonstration.

11. Organisation de l'équipe

11.1 Répartition des modules et des responsabilités

Développeur	Module (15 j-h)	Responsabilité transverse (3 j-h)	Total
Dev 1	M1 : Auth et équipes	Bibliothèque de permissions	18 j-h
Dev 2	M2 : Kanban	Socle d'interface (coque, design system)	18 j-h
Dev 3	M3 : Tâches et assignations	Schéma de données et migrations	18 j-h
Dev 4	M4 : Calendrier, statistiques et rapports	Journal d'activité	18 j-h
Dev 5	M5 : Chat et notifications	Temps réel et tâches planifiées (serveur)	18 j-h
Dev 6	M6 : Fichiers	Intégration continue et sécurité	18 j-h
Dev 7	M7 : Formations et coaching	Tests de bout en bout	18 j-h

Un chef de projet (rotatif toutes les 2 semaines, ou désigné) anime les réunions et arbitre les désaccords. Il peut être l'un des 7 développeurs.

11.2 Workflow Git

- Dépôt unique GitHub, branche main protégée (fusion uniquement par pull request).
- Branche d'intégration develop ; branches de fonctionnalité feat/m4-calendar-sources, fix/m2-drag-bug.

- Pull request obligatoire avec au moins 1 relecture (idéalement croisée avec le module voisin), CI au vert, description claire.
- Commits suivant Conventional Commits (feat:, fix:, docs:, refactor:, test:).
- Les modifications d'un .spy.yaml ou d'une migration sont annoncées avant fusion, relues par le Dev 3, et les migrations fusionnées rapidement.
- Le code généré par Serverpod est versionné ; après chaque fusion sur develop, exécuter serverpod generate. La CI refuse une pull request dont le code généré n'est pas à jour.

11.3 Rituels

- Réunion de lancement (semaine 0) : validation du cahier des charges, choix du nom et du design, atelier Dart / Flutter.
- Point court 2 fois par semaine (15 min).
- Revue de fin de sprint (toutes les 2 semaines) : démonstration et suivi de la charge (§8.4).
- Tableau de suivi : StudyFlow lui-même dès qu'il est utilisable (dogfooding).

11.4 Définition de « terminé »

Une fonctionnalité est terminée lorsque :

1. Elle respecte les critères d'acceptation du module.
2. Le code est relu et fusionné via pull request.
3. Les permissions sont vérifiées côté serveur.
4. Elle est utilisable sur mobile (375 px) et au clavier.
5. Les cas d'erreur et les états vides sont gérés.
6. Un test existe pour la logique critique.
7. La documentation du module est à jour.

12. Planning prévisionnel

Planning indicatif sur 8 semaines (à adapter à votre calendrier réel).

Semaine	Phase	Contenu
S0	Cadrage	Validation du cahier des charges, atelier Dart / Flutter / Riverpod (2 jours), maquettes (Figma), choix du nom, création du dépôt, CI, conventions, version de Flutter figée
S1	Socle	Projet Serverpod et application Flutter, Docker Compose, **24 modèles** et migrations (Dev 3), authentification et permissions dont le rôle Coach (Dev 1), coque et design system (Dev 2), journal d'activité et interface CalendarSource (Dev 4), temps réel et appels futurs (Dev 5). Prototypes Kanban (M2) et calendrier (M4). Les modules démarrent avec des données fictives. Revue des estimations en fin de semaine
S2	Sprint 1	Fonctionnalités P1 de chaque module (6 par module, 8 j-h)
S3	Sprint 1 (fin)	Fin des P1. Première intégration : « créer équipe → projet → tâche → déplacer », et « publier une formation → s'inscrire → la voir au calendrier ». Démo interne et point de suivi
S4	Sprint 2	Fonctionnalités P2 de chaque module (4 par module, 5 j-h)
S5	Sprint 2 (fin)	Fin des P2, notifications, filtres, première version des rapports. Deuxième intégration. Revue de code croisée et point de suivi
S6	Consolidation	Bugs, performance (DevTools), accessibilité, responsive, audit des permissions et de la sécurité (dont notes privées de coaching), tests de bout en bout
S7	Finalisation	Fonctionnalités P3 (2 par module, 2 j-h), jeu de données de démo, documentation, préparation de la démonstration (compilation Android en bonus)
S8	Livraison	Gel du code, répétition de la soutenance, rapport final, démonstration

Jalons clés

Jalon	Échéance	Critère
J1	Fin S1	Socle opérationnel en local, tous les modules branchés sur la base
J2	Fin S3	Parcours principal fonctionnel de bout en bout, tous les P1 livrés
J3	Fin S5	Toutes les fonctionnalités P1 et P2 intégrées
J4	Fin S7	Application stable, documentée, démonstration prête
J5	S8	Soutenance

> **Conseil.** Ne laissez pas l'intégration à la fin. Chaque sprint doit se terminer par une application qui tourne avec tous les modules ensemble, même incomplets.

13. Livrables

1. **Code source** : dépôt GitHub propre (client, serveur, package partagé), historique lisible, README complet.
2. **Application exécutable en local** : guide d'installation, scripts de lancement (docker compose up -d, tool/dev.sh) et identifiants de démonstration (dont un compte Coach).
3. **Base de données** : modèles .spy.yaml, migrations, script de données de démonstration.
4. **Documentation technique** : architecture, modèle de données (diagramme), guide d'installation, une page par module.
5. **Documentation utilisateur** : guide de prise en main (captures d'écran ou courte vidéo), avec un chapitre formations et coaching.
6. **Maquettes** : fichier Figma ou équivalent.
7. **Rapport de projet** : organisation, difficultés, choix techniques, bilan individuel par module (jours estimés et réalisés).
8. **Présentation de soutenance** : support et démonstration en direct (prévoir une vidéo de secours).
9. **Rapport de tests** : tests automatisés (unitaires, widgets, bout en bout) et résultats.

14. Risques et parades

Risque	Probabilité	Impact	Parade
Équipe peu familière avec Dart, Flutter et Riverpod	Élevée	Élevé	Atelier de 2 jours en S0, binômes croisés, code de référence (module d'exemple), revues précoces
Conflits Git fréquents (dont code généré)	Élevée	Moyen	Un dossier par module, un .spy.yaml par entité, branches courtes, serverpod generate après fusion, vérification en CI
Retard d'un module bloquant les autres (M1, M3, M4)	Moyenne	Élevé	Livrer permissions, modèles, journal d'activité et CalendarSource en S1 ; les autres travaillent avec des dépôts simulés
Changements du schéma mal coordonnés	Élevée	Élevé	Dev 3 responsable des modèles et migrations, annonce préalable, migrations fusionnées vite
Densité du M4 (calendrier + statistiques + journal)	Moyenne	Élevé	Démarrer par le calendrier et les agrégations simples, graphiques avancés en P2, transfert de P2/P3 en cas d'écart (§8.4)
Fuite des notes privées de coaching	Faible	Élevé	Vérification serveur par requireCoach, champ jamais sérialisé hors du coach auteur, tests dédiés, audit en S6
Dépassement de capacité en cas d'inscriptions simultanées	Moyenne	Moyen	Transaction SQL, test de concurrence, index unique trainingId + userId
Dépendance du M7 envers le M4 (sources du calendrier)	Moyenne	Moyen	Interface CalendarSource figée en S1 ; le M7 teste avec une source simulée

Risque	Probabilité	Impact	Parade
Confusion entre rôles (Coach, Observateur, Membre)	Moyenne	Moyen	Matrice testée unitairement, libellés explicites dans l'interface, scénario de recette dédié
Bibliothèques Kanban ou calendrier insuffisantes	Moyenne	Moyen	Prototype d'une journée en S1 ; repli sur Draggable / DragTarget et calendrier maison avec intl
Flutter Web lourd ou peu accessible	Moyenne	Moyen	Build release pour la démo, arbre sémantique activé, mesures DevTools en S6
Environnement local différent d'une machine à l'autre	Moyenne	Moyen	Docker Compose, FVM, README testé sur deux machines
Intégration tardive révélant des incohérences	Élevée	Élevé	Intégration à chaque fin de sprint, démo commune
Faille de sécurité (accès aux données d'une autre équipe)	Moyenne	Élevé	Fonctions de permission centralisées, tests dédiés, audit en S6
Charge inégale, surcharge ou abandon d'un membre	Moyenne	Moyen	Estimations en j-h, suivi à chaque sprint, transfert de P2/P3, documentation et binômage croisé
Données de démonstration perdues ou incohérentes	Faible	Moyen	Script reproductible, vidéo de secours
Interface incohérente d'un module à l'autre	Élevée	Moyen	Design system partagé, maquettes validées, relecture UI en revue de sprint
Fonctionnalités trop ambitieuses (P3 avant P1)	Moyenne	Moyen	Respect strict de l'ordre de priorité, validation du MVP avant tout bonus

15. Critères de validation

15.1 Recette fonctionnelle (scénario de démonstration)

1. Créer un compte, se connecter, modifier son profil. (M1)
2. Créer une équipe, inviter deux personnes par lien et un coach avec le rôle Coach. (M1)
3. Créer un projet, personnaliser les colonnes. (M2)
4. Créer 5 tâches, les assigner, ajouter étiquettes, sous-tâches et échéances. (M3)
5. Déplacer des cartes par glisser-déposer, vérifier que l'autre navigateur se met à jour. (M2, M5)
6. Retrouver ces échéances dans le calendrier, créer un jalon. (M4)
7. Échanger des messages à deux dans le chat, voir la notification d'assignation. (M5)
8. Déposer un fichier, le joindre à une tâche, le prévisualiser, le télécharger. (M6)
9. Consulter le tableau de bord, générer et exporter le rapport PDF. (M4)
10. Le coach publie une formation ; un membre s'inscrit et la voit dans son calendrier ; une inscription au-delà de la capacité passe en liste d'attente. (M7, M4)
11. Un membre demande une séance de coaching ; le coach confirme un créneau, rédige un compte rendu partagé et une note privée ; le coaché voit le compte rendu mais pas la note. (M7, M5)
12. Vérifier qu'un observateur ne peut rien modifier, qu'un coach ne peut pas modifier une tâche, et qu'un non-membre n'accède à rien.

15.2 Barème indicatif de qualité

Critère	Poids indicatif
Fonctionnalités P1 complètes et fonctionnelles	30 %
Qualité du code, architecture, tests	20 %
Intégration réussie des 7 modules	15 %
Sécurité et gestion des permissions	10 %
Ergonomie, design, accessibilité	10 %
Documentation et rapport	10 %
Soutenance et démonstration	5 %

(À ajuster selon les consignes de votre enseignant.)

16. Annexes

16.1 Liste indicative des endpoints et routes

Module	Méthodes d'endpoint / routes principales
M1	register, updateProfile, createTeam, manageTeam, regenerateInviteLink, acceptInvitation, updateMemberRole, removeMember
M2	createProject, updateProject, listProjects, createColumn, updateColumn, deleteColumn, reorderColumns, moveTask
M3	createTask, updateTask, deleteTask, duplicateTask, setAssignees, saveLabel, saveSubtask, listTasks
M4	listCalendarEvents, listMyTasks, saveMilestone, getProjectStats, getBurndown, listActivity, generateReport, exportReportPdf. Complémentaires : deleteMilestone, updateDueDate, getMemberWorkload, getVelocity, exportCsv, regenerateCalendarToken, route GET /calendar/<token>.ics, appels futurs sendDueReminders et takeDailySnapshot
M5	sendMessage, editMessage, deleteMessage, listMessages, markChannelRead, getTaskChannel, listNotifications, markNotificationRead (plus notify() et le flux watchChannel)
M6	route POST /upload, listFiles, getDownloadUrl, renameFile, moveFile, deleteFile, saveFolder, replaceFile

Module	Méthodes d'endpoint / routes principales
M7	listTrainings, saveTraining, enrollInTraining, markAttendance, requestCoaching, respondToCoaching, saveSessionReport, saveGoal. Complémentaires : listMySessions, saveSkillLevel, listSkillLevels, submitFeedback, saveTrainingResource, appel futur sendTrainingReminders

16.2 Configuration et variables

Élément	Emplacement	Valeur par défaut / rôle
Base de données (hôte, port, nom)	studyflow_server/config/development.yaml	localhost:8090, base studyflow (PostgreSQL de Docker Compose)
Mot de passe de la base, secret de service	config/passwords.yaml (non versionné)	À générer ; modèle dans passwords.yaml.example
Secrets de l'authentification	config/passwords.yaml	À générer, jamais dans Git
Identifiants OAuth Google / GitHub	config/passwords.yaml	Facultatifs (fonctionnalité 1.8)
API_URL	Option --dart-define de flutter run	http://localhost:8080/
STUDYFLOW_UPLOAD_DIR	Variable d'environnement du serveur	./uploads
STUDYFLOW_MAX_UPLOAD_MB	Variable d'environnement du serveur	20
STUDYFLOW_TEAM_QUOTA_MB	Variable d'environnement du serveur	500
STUDYFLOW_SMTP	Variable d'environnement du serveur	localhost:1025 (Mailpit) ; expéditeur studyflow@localhost
STUDYFLOW_TRAINING_REMINDER_MIN	Variable d'environnement du serveur	60 : délai du rappel avant une formation ou une séance

16.3 Correspondance avec la version 2.0

Version 2.0	Version 2.1
M4 : Calendrier et échéances	M4 : Calendrier, échéances, statistiques et rapports (6 entités)
M7 : Statistiques et rapports	Fusionné dans le M4 (fonctions 7.1 à 7.12 regroupées en 4.5, 4.6, 4.9, 4.10, 4.11, 4.12)

Version 2.0	Version 2.1
Pas de M7 de formation	M7 : Formations et coaching (Training, CoachingSession, CoachingGoal)
4 rôles	5 rôles (ajout de Coach)
21 entités	24 entités
logActivity défini par le M7	logActivity défini par le M4 (Dev 4)
Tests de bout en bout par le Dev 4 (5 scénarios)	Par le Dev 7 (7 scénarios)
Documentation et seed par le Dev 7	Répartis : pages par module, seed assemblé par le Dev 7, README par le Dev 1

16.4 Glossaire

Terme	Définition
Flutter	Boîte à outils d'interface de Google : une seule base de code pour le web, Android, iOS et le bureau
Dart	Langage de Flutter, typé, avec null safety, utilisable aussi côté serveur
Widget	Brique de base d'une interface Flutter
Riverpod	Bibliothèque de gestion d'état et d'injection de dépendances
go_router	Bibliothèque de navigation déclarative, gérant URL et liens profonds
Serverpod	Serveur d'applications écrit en Dart, avec ORM, authentification, flux temps réel et tâches planifiées
Endpoint	Classe du serveur dont les méthodes publiques sont appelables depuis le client Flutter
Appel futur (FutureCall)	Tâche exécutée par Serverpod à une date ou après un délai donnés

Terme	Définition
Stream	Flux de valeurs asynchrones en Dart ; sert ici au temps réel
Kanban	Méthode visuelle de suivi du travail en colonnes
WIP limit	Nombre maximum de tâches simultanées autorisées dans une colonne
Burndown chart	Graphique du travail restant au fil du temps
Vélocité	Quantité de travail terminée par unité de temps
Jalon (milestone)	Date clé du projet (rendu, soutenance)
CalendarSource	Interface du M4 que chaque module implémente pour fournir des événements au calendrier
Coach	Rôle d'équipe : enseignant, mentor ou ancien qui anime des formations et accompagne des membres
Coaching (séance)	Entretien individuel entre un coach et un membre, avec compte rendu et actions
Notes privées	Notes d'un coach sur une séance, lisibles uniquement par lui
Liste d'attente	Inscription en surnombre, promue automatiquement en cas de désistement
Compétence (SkillLevel)	Niveau de 0 à 5 d'un membre sur une compétence, auto-évalué ou évalué par un coach
Jour-homme (j-h)	Unité de charge : une journée de travail effectif d'une personne
Responsabilité transverse	Travail confié à un seul développeur mais utile à toute l'équipe
Mailpit	Serveur d'e-mails local qui capture les messages de test
Fractional indexing	Ordonnancement permettant d'insérer un élément entre deux autres sans renuméroter

Terme	Définition
Mise à jour optimiste	Mise à jour de l’interface avant la confirmation du serveur, avec retour arrière en cas d’échec
Soft delete	Suppression logique : la donnée est marquée supprimée mais conservée
RGPD	Règlement général sur la protection des données
DoD	Definition of Done, critères de fin d’une fonctionnalité